GPUMODE Lecture Study Notes: Lecture 47 Kernelbot Benchmark

Core Problem the Lecture Addresses

This lecture introduces a public GPU kernel leaderboard integrated into the GPU Mode Discord community. The central problem is not merely that people need to learn CUDA, Triton, or GPU programming syntax. The deeper problem is that writing a kernel that *works* is very different from writing a kernel that is *fast, correct, benchmarkable, reproducible, and useful as training data for future systems.*

> The lecture frames GPU kernel writing as a competitive programming problem for modern accelerators: give people a well-specified operator, fixed input shapes, public correctness tests, private benchmark runs, and real GPU hardware, then let the community discover increasingly optimized implementations.

The speakers emphasize several bottlenecks in the current GPU kernel ecosystem:

- Many people can now learn the basics of CUDA, Triton, PyTorch extensions, and GPU architecture, but they lack realistic opportunities to apply those skills on real hardware.
- High-performance production kernels are often locked inside companies and cannot be open sourced.
- Public repositories contain relatively few highly optimized kernels compared with the demand for learning material.
- AI-generated GPU kernels are a growing research direction, but evaluating them requires robust benchmarks, correctness checks, and timing methodology.
- Kernel performance is hardware-specific. A solution that is excellent on an H100 may not be optimal on a T4, L4, A100, or MI300.
- Measurement noise, cloud GPU variability, thermal behavior, power limits, and benchmarking harness design can distort leaderboard results.

The leaderboard is therefore designed as both a learning platform and an evaluation platform. It lets humans and eventually AI agents submit kernels, receive correctness and benchmark feedback, and compare against other implementations on real GPUs.

Required Background

GPU Kernel Programming

A GPU kernel is a function launched over many parallel threads. In CUDA terminology, threads are grouped into thread blocks, and blocks form a grid. Threads execute on streaming multiprocessors, usually abbreviated as SMs.

For an operation such as vector addition,

$$c_i = a_i + b_i,$$

a simple kernel assigns one thread to one element i. The indexing logic is usually:

$$i = \texttt{blockIdx.x} \cdot \texttt{blockDim.x} + \texttt{threadIdx.x}.$$

This mapping is the first level of GPU programming: mapping data elements to parallel threads.

However, high-performance GPU programming requires more than parallel indexing. It requires understanding:

- memory coalescing,
- occupancy,
- register pressure,
- shared memory tiling,
- warp-level execution,
- tensor core utilization,
- memory bandwidth,
- arithmetic intensity,
- launch overhead,
- synchronization,
- hardware-specific instructions.

From Correct Kernels to Fast Kernels

A correct kernel produces the right output within a specified tolerance. A fast kernel minimizes runtime while preserving correctness. The lecture repeatedly emphasizes that these are different milestones.

A correctness-oriented implementation may simply call PyTorch:

```
import torch

def custom_kernel(a, b):
    return torch.matmul(a, b)
```

This is likely correct and often already fast because PyTorch dispatches to heavily optimized libraries such as cuBLAS, cuDNN, or vendor-specific backends. But it does not teach the submitter how the kernel works internally.

A learning-oriented custom kernel may be slower than PyTorch but still valuable because it exposes the implementation details.

A leaderboard-oriented kernel tries to beat or approach the reference implementation by exploiting:

- known input shapes,
- known data distributions,
- specific device properties,
- problem-specific invariants,
- lower-level CUDA, PTX, or Triton features,
- benchmark harness behavior.

Competitive Programming Analogy

The speakers compare GPU kernel optimization to competitive programming. In competitive programming, a participant receives a formal problem statement, input constraints, hidden tests, and a scoring or pass/fail system. The goal is to produce the fastest or most efficient algorithm under those constraints.

The GPU leaderboard mirrors this structure:

- Each task defines an operator, input shape, output shape, and correctness criteria.
- Each submission is tested on public or semi-public cases.
- Benchmark cases determine the ranking.
- Different hardware targets are treated as separate leaderboards.
- At the end of a competition, strong solutions can be released as a public dataset.

Main Concepts

The Purpose of the Kernel Leaderboard

The leaderboard exists to create a public space where people can practice GPU kernel writing on actual devices without needing to own expensive GPUs.

The lecture highlights that few people have access to devices such as H100, A100, L4, T4, or MI300 machines. The leaderboard reduces this barrier by running submissions on externally provisioned GPUs through infrastructure such as Modal and hardware sponsors.

The major goals are:

1. Make GPU kernel writing accessible.
2. Give learners a concrete benchmark for their progress.
3. Encourage open sharing of optimized kernels.
4. Create public datasets of human-written high-performance kernels.
5. Improve evaluation methods for AI-generated GPU code.
6. Let hardware vendors or companies submit custom kernels as benchmark problems.

KernelBench and AI-Generated Kernels

The lecture connects the leaderboard to the broader research direction of evaluating AI-generated GPU kernels. The speakers refer to recent work around benchmark suites for AI-generated CUDA code.

The core research question is:

Can an AI system generate GPU kernels that are correct and competitive with human-written implementatio

This question has three subproblems:

1. **Synthesis:** Can the model write code with the correct operator semantics?
2. **Compilation:** Does the generated code compile and run on the target hardware?
3. **Performance:** Does it achieve meaningful speed relative to a strong baseline?

The leaderboard is useful because it can produce examples of optimized kernels and also expose failure modes in correctness tests. If a human can reward-hack the evaluation harness, an AI agent may discover similar hacks automatically.

Practice Round

The initial round is a practice round. Its purpose is to test the infrastructure and gather feedback, not to create final training data.

Important properties of the practice round include:

- Submissions are intended to test the Discord bot, Modal execution flow, leaderboard storage, and reporting.
- The practice round uses Python modules as the primary submission format.
- Inline CUDA, Triton, PyTorch, and theoretically other Python-accessible GPU DSLs can be used from inside the Python file.
- The supported devices include T4, L4, A100, and H100, with plans for MI300 and additional hardware.
- Each problem-device pair is a separate leaderboard.

If there are P problems and D devices, the effective number of leaderboards is:

$$L = P \cdot D.$$

For example, with $P = 8$ practice problems and $D = 4$ devices,

$$L = 8 \cdot 4 = 32.$$

Problem-Device Separation

A key design choice is that each operator on each GPU is graded separately. Matrix multiplication on a T4 is not the same leaderboard as matrix multiplication on an H100.

This matters because hardware differences strongly affect optimization strategy. For example:

- H100 has newer tensor cores, higher memory bandwidth, and different scheduling behavior than T4.
- A100 and H100 support different peak throughput regimes.
- L4 and T4 are more constrained devices where memory bandwidth, occupancy, and launch overhead may dominate.
- MI300 introduces AMD-specific programming and hardware behavior.

The optimization objective is therefore not universal performance portability. Instead, the leaderboard rewards device-specific specialization.

Known Shapes and Hyper-Optimization

The lecture explicitly encourages participants to optimize for the known shapes in the problem definition.

This is important because many production kernels are fast precisely because they exploit fixed or narrow shape regimes. For example, FlashAttention can be highly optimized because transformer attention often uses predictable head dimensions such as:

$$d_{\text{head}} = 64 \quad \text{or} \quad d_{\text{head}} = 128.$$

If shapes are fixed, a kernel can specialize:

- tile sizes,
- block dimensions,
- shared memory layout,
- vectorized load width,
- unroll factors,
- register blocking,
- tensor core fragment shapes,
- boundary checks.

A generic PyTorch kernel must work over many shapes and many layouts. A leaderboard kernel can use much stronger assumptions.

Hardware-Level Explanation

Why Hardware-Specific Leaderboards Matter

GPU performance depends on how the kernel maps to the hardware. A kernel's runtime can be approximated by the maximum of compute time, memory time, and overhead time:

$$T_{\text{kernel}} \approx \max\left(T_{\text{compute}}, T_{\text{memory}}\right) + T_{\text{launch}} + T_{\text{sync}}.$$

The compute term is approximately:

$$T_{\text{compute}} = \frac{\text{FLOPs}}{\text{Sustained FLOPs/s}},$$

and the memory term is approximately:

$$T_{\text{memory}} = \frac{\text{Bytes moved}}{\text{Sustained bandwidth}}.$$

Different GPUs have different sustained FLOP rates, memory bandwidth, cache sizes, tensor core capabilities, and scheduling behavior. Therefore, a kernel that is compute-bound on one GPU may be memory-bound on another.

Streaming Multiprocessors and Warps

An NVIDIA GPU contains multiple SMs. Each SM schedules warps, where a warp is typically 32 threads executing in SIMT style. A kernel launch creates a grid of thread blocks, and blocks are assigned to SMs.

The occupancy of a kernel is constrained by:

- number of registers per thread,
- shared memory per block,
- maximum threads per block,
- maximum resident blocks per SM,
- maximum resident warps per SM.

A simplified occupancy estimate is:

$$\text{Occupancy} = \frac{\text{Active warps per SM}}{\text{Maximum warps per SM}}.$$

High occupancy can help hide latency, but it is not the only goal. For highly optimized tensor-core kernels, lower occupancy may still be acceptable if each warp performs high-throughput work and memory reuse is strong.

Memory Coalescing

Global memory access is efficient when adjacent threads in a warp access adjacent memory addresses. For a warp of 32 threads reading `float32` values, the ideal contiguous access covers:

$$32 \cdot 4 = 128 \text{ bytes}.$$

If thread t reads element $i + t$, the memory access is coalesced. If thread t reads a strided or random location, the hardware may need multiple memory transactions.

For leaderboard kernels, coalescing is crucial for tasks such as:

- vector sum,
- elementwise operations,
- reductions,
- matrix multiplication global loads,
- convolution input reads,

- prefix sum and scan.

Shared Memory and Tiling

Many fast kernels use shared memory to reuse data loaded from global memory. For matrix multiplication,

$$C = AB,$$

where $A \in \mathbb{R}^{M \times K}$, $B \in \mathbb{R}^{K \times N}$, and $C \in \mathbb{R}^{M \times N}$, a tiled kernel loads subtiles of A and B into shared memory.

The scalar definition is:

$$C_{ij} = \sum_{k=0}^{K-1} A_{ik} B_{kj}.$$

A tiled implementation splits the K-dimension into chunks of size T_K:

$$C_{ij} = \sum_{q=0}^{K/T_K - 1} \sum_{r=0}^{T_K - 1} A_{i,qT_K+r} B_{qT_K+r,j}.$$

This increases arithmetic intensity because each loaded tile element participates in many multiply-add operations.

Tensor Cores

For matrix multiplication and convolution, tensor cores can provide much higher throughput than scalar CUDA cores. Tensor cores operate on small matrix fragments and perform matrix multiply-accumulate operations:

$$D = AB + C.$$

A leaderboard matmul kernel may need to choose whether to use:

- FP32 CUDA cores,
- TF32 tensor cores,
- FP16 tensor cores,
- BF16 tensor cores,
- INT8 tensor cores,
- mixed-precision accumulation.

However, the lecture notes that correctness tolerances matter. If the benchmark expects FP32-level accuracy, simply casting to BF16 may fail. TF32 may sometimes pass depending on tolerance and input distribution, but this must be verified.

Mathematical Reasoning

Leaderboard Score

For the practice round, execution time is the primary metric. If a benchmark runs a kernel R times and records times $t_1, t_2, \ldots, t_R$, the mean runtime is:

$$\bar{t} = \frac{1}{R} \sum_{r=1}^{R} t_r.$$

The variance is:

$$s^2 = \frac{1}{R-1} \sum_{r=1}^{R} \left(t_r - \bar{t}\right)^2.$$

The standard error of the mean is:

$$\text{SEM} = \frac{s}{\sqrt{R}}.$$

The leaderboard score can be understood as a function of measured runtime:

$$\text{Score} = f(\bar{t}),$$

where lower runtime is better. In the practice round, the score is essentially speed.

Speedup

If T_{ref} is the reference runtime and T_{sub} is the submitted kernel runtime, speedup is:

$$S = \frac{T_{\text{ref}}}{T_{\text{sub}}}.$$

If $S > 1$, the submission is faster than the reference. If $S < 1$, it is slower.

The lecture cautions that PyTorch references are already highly optimized. Therefore, achieving even 80% of PyTorch speed can indicate a solid custom kernel:

$$\frac{T_{\text{ref}}}{T_{\text{sub}}} \approx 0.8$$

may still be respectable if the reference calls a mature vendor library.

Arithmetic Intensity

Arithmetic intensity is:

$$\text{Arithmetic Intensity} = \frac{\text{FLOPs}}{\text{Bytes moved}}.$$

A kernel is likely memory-bound when:

$$\frac{\text{FLOPs}}{\text{Bytes moved}} < \frac{\text{Peak FLOPs/s}}{\text{Peak bytes/s}}.$$

It is likely compute-bound when:

$$\frac{\text{FLOPs}}{\text{Bytes moved}} > \frac{\text{Peak FLOPs/s}}{\text{Peak bytes/s}}.$$

This is the roofline model. It explains why vector operations are often bandwidth-bound while matrix multiplication can become compute-bound when tiling reuses data effectively.

Vector Sum Reward Hacking

The lecture gives a concrete example of correctness-test design. Suppose a task asks for:

$$y = \sum_{i=1}^{n} x_i.$$

If each x_i is sampled independently from a normal distribution:

$$x_i \sim \mathcal{N}(0, 1),$$

then:

$$\mathbb{E}[y] = \sum_{i=1}^{n} \mathbb{E}[x_i] = 0.$$

The variance is:

$$\text{Var}(y) = \sum_{i=1}^{n} \text{Var}(x_i) = n.$$

For very large n, the sum is not exactly zero, but relative to some tolerance choices an implementation that always returns zero may pass too often. This is a reward-hacking failure mode. A robust test must avoid distributions that allow trivial approximations.

One strategy is to sample with a random offset and scale:

$$x_i = \alpha z_i + \beta, \quad z_i \sim \mathcal{N}(0, 1).$$

Then:

$$\mathbb{E}\left[\sum_{i=1}^{n} x_i\right] = n\beta.$$

If $\beta \neq 0$, the always-zero solution fails.

Floating-Point Tolerance

Correctness is usually checked using absolute and relative tolerances:

$$|y_{\text{sub}} - y_{\text{ref}}| \leq \text{atol} + \text{rtol} \cdot |y_{\text{ref}}|.$$

Here:

- y_{sub} is the submitted output,
- y_{ref} is the reference output,
- atol is absolute tolerance,
- rtol is relative tolerance.

For reductions and prefix sums, error may grow with problem size. A fixed tolerance may be too strict for large n or too loose for small n. A more robust tolerance may depend on n:

$$\text{atol}(n) = c_1 + c_2\sqrt{n}\epsilon,$$

where ϵ is machine precision and c_1, c_2 are problem-dependent constants.

Code Walkthrough

Submission Interface

The leaderboard expects a Python file that defines a function with the correct interface. A minimal matmul-style submission may look like this:

```
import torch

def custom_kernel(a: torch.Tensor, b: torch.Tensor) -> torch.Tensor:
    return torch.matmul(a, b)
```

This submission is simple but important for understanding the workflow:

- The function receives tensors generated by the evaluation harness.
- The function returns a tensor with the expected output shape.
- The correctness checker compares the returned tensor against a reference implementation.
- The benchmark runner measures runtime over selected benchmark cases.

Hardware behavior:

- The call to `torch.matmul` dispatches to a backend implementation.
- On NVIDIA GPUs this may use cuBLAS or another optimized path.

- The actual kernel may use tensor cores depending on dtype, shape, and backend settings.
- The user does not directly control thread layout, shared memory tiling, or register blocking.

Inline CUDA from Python

The lecture notes that direct CUDA submissions are still being tested, but inline CUDA inside a Python submission is supported. A representative pattern is:

```
import torch
from torch.utils.cpp_extension import load_inline

cuda_source = r"""
extern "C" __global__
void vector_add_kernel(const float* a,
                       const float* b,
                       float* c,
                       int n) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx < n) {
        c[idx] = a[idx] + b[idx];
    }
}
"""

module = load_inline(
    name="vector_add_ext",
    cpp_sources="",
    cuda_sources=cuda_source,
    functions=["vector_add_kernel"],
    verbose=False,
)

def custom_kernel(a: torch.Tensor, b: torch.Tensor) -> torch.Tensor:
    n = a.numel()
    c = torch.empty_like(a)
    threads = 256
    blocks = (n + threads - 1) // threads
    module.vector_add_kernel(
        grid=(blocks,),
        block=(threads,),
        args=[a, b, c, n],
    )
    return c
```

The indexing math is:

$$\text{idx} = \texttt{blockIdx.x} \cdot \texttt{blockDim.x} + \texttt{threadIdx.x}.$$

The number of blocks is:

$$B = \left\lceil \frac{n}{T} \right\rceil,$$

where T is the number of threads per block.

The code uses:

$$B = \frac{n + T - 1}{T}$$

with integer division.

Hardware explanation:

- Each thread computes one output element.
- Consecutive threads read consecutive elements of `a` and `b`.
- The global memory loads are coalesced when tensors are contiguous.
- The kernel is bandwidth-bound because each element performs one addition but moves three floating-point values: two reads and one write.

For FP32 vector addition, bytes moved per element are:

$$2 \cdot 4 + 1 \cdot 4 = 12 \text{ bytes}.$$

FLOPs per element are:

$$1.$$

Therefore arithmetic intensity is:

$$\frac{1}{12} \text{ FLOPs/byte}.$$

This is very low, so the operation is memory-bound.

Triton Submission Pattern

Triton is convenient because it allows writing GPU kernels in Python while exposing program IDs, vectorized blocks, masks, and explicit memory operations.

A representative vector addition Triton submission is:

```
import torch
import triton
import triton.language as tl

@triton.jit
def add_kernel(a_ptr, b_ptr, c_ptr, n: tl.constexpr, BLOCK: tl.constexpr):
```

```
    pid = tl.program_id(axis=0)
    offsets = pid * BLOCK + tl.arange(0, BLOCK)
    mask = offsets < n

    a = tl.load(a_ptr + offsets, mask=mask, other=0.0)
    b = tl.load(b_ptr + offsets, mask=mask, other=0.0)
    c = a + b

    tl.store(c_ptr + offsets, c, mask=mask)

def custom_kernel(a: torch.Tensor, b: torch.Tensor) -> torch.Tensor:
    c = torch.empty_like(a)
    n = a.numel()
    block = 1024
    grid = (triton.cdiv(n, block),)
    add_kernel[grid](a, b, c, n, BLOCK=block)
    return c
```

The Triton program ID acts like a CUDA block index:

$$\text{pid} = \texttt{tl.program\_id}(0).$$

Each Triton program handles a vector of offsets:

$$\text{offsets} = \text{pid} \cdot \text{BLOCK} + [0, 1, \ldots, \text{BLOCK} - 1].$$

The mask prevents out-of-bounds memory access:

$$\text{mask}_j = (\text{offsets}_j < n)\,.$$

Hardware behavior:

- Triton maps vector operations to GPU threads and warps.
- The compiler chooses a lowering strategy based on block size and target GPU.
- The contiguous offsets encourage coalesced memory access.
- The mask introduces boundary handling but avoids a separate cleanup kernel.

Discord Command Workflow

The lecture demonstrates a Discord bot interface. A participant can list available leaderboards, show a specific leaderboard, run tests, benchmark a submission, and submit a ranked result.

Representative commands are:

```
/leaderboard list
/leaderboard show leaderboard_name:conv2d
/leaderboard submit test leaderboard_name:conv2d gpu:h100 file:submission.py
/leaderboard submit benchmark leaderboard_name:conv2d gpu:h100 file:submission.py
/leaderboard submit ranked leaderboard_name:conv2d gpu:h100 file:submission.py
```

The intended workflow is:

1. Use `/leaderboard list` to inspect available tasks and deadlines.
2. Use `/leaderboard show` to see current rankings.
3. Use `/leaderboard submit test` to check correctness quickly.
4. Use `/leaderboard submit benchmark` to measure speed without ranking.
5. Use `/leaderboard submit ranked` to run correctness and benchmark checks, then post the result to the leaderboard.

Task Specification

The task definition is driven by a configuration file. The lecture describes a YAML-style task file that specifies:

- task name,
- description,
- input and output types,
- test cases,
- benchmark cases,
- device options,
- optional template files.

A representative task specification is:

```
name: matmul
description: Matrix multiplication C = A @ B

tests:
  - m: 256
    n: 256
    k: 256
    seed: 0
  - m: 512
    n: 512
    k: 512
    seed: 1
```

```
benchmarks:
  - m: 4096
    n: 4096
    k: 4096
    seed: 42

devices:
  - t4
  - l4
  - a100
  - h100
```

This task specification creates concrete test cases. For matmul, a test specification contains:

$$(M, N, K, \text{seed}).$$

The input tensors are:

$$A \in \mathbb{R}^{M \times K}, \quad B \in \mathbb{R}^{K \times N}.$$

The output tensor is:

$$C \in \mathbb{R}^{M \times N}.$$

Reference Kernel and Input Generation

The reference implementation generates inputs, computes the expected output, and checks the submitted output.

A representative reference file is:

```python
import torch
from typing import TypedDict, Tuple

class TestSpec(TypedDict):
    m: int
    n: int
    k: int
    seed: int

Input = Tuple[torch.Tensor, torch.Tensor]
Output = torch.Tensor

def generate_input(spec: TestSpec) -> Input:
    torch.manual_seed(spec["seed"])
    a = torch.rand((spec["m"], spec["k"]), device="cuda", dtype=torch.float32)
    b = torch.rand((spec["k"], spec["n"]), device="cuda", dtype=torch.float32)
    return a, b
```

```
def reference_call(inp: Input) -> Output:
    a, b = inp
    return torch.matmul(a, b)

def check_implementation(actual: Output, expected: Output) -> None:
    torch.testing.assert_close(
        actual,
        expected,
        rtol=1e-3,
        atol=1e-3,
    )
```

Important design decisions:

- The input generator controls data distribution.
- The seed makes tests reproducible.
- The reference call defines semantic correctness.
- The tolerance defines acceptable numerical deviation.
- A verbose close-checker is useful because it reports shape mismatch, dtype mismatch, and numerical mismatch separately.

Evaluation Harness

The evaluation harness connects the submitted code, generated inputs, reference output, correctness checker, and benchmark timer.

A representative evaluation harness is:

```
import time
import torch

def synchronize():
    torch.cuda.synchronize()

def time_call(fn, args, warmup=5, repeat=50):
    for _ in range(warmup):
        out = fn(*args)
    synchronize()

    times = []
    for _ in range(repeat):
        start = time.perf_counter()
        out = fn(*args)
        synchronize()
        end = time.perf_counter()
```

```
        times.append(end - start)

    return out, times

def evaluate_submission(custom_kernel, specs, mode):
    results = []

    for spec in specs:
        inp = generate_input(spec)
        expected = reference_call(inp)

        actual, times = time_call(custom_kernel, inp)

        check_implementation(actual, expected)

        results.append({
            "spec": spec,
            "mean": sum(times) / len(times),
            "min": min(times),
            "max": max(times),
        })

    return results
```

A critical detail is `torch.cuda.synchronize`. GPU kernels are asynchronous with respect to the CPU. Without synchronization, a CPU timer may only measure launch overhead rather than actual kernel execution time.

The correct timing pattern is:

$$\text{launch kernel} \rightarrow \text{synchronize} \rightarrow \text{record elapsed time.}$$

Performance Intuition

Why PyTorch is Hard to Beat

PyTorch often calls highly optimized libraries. For common operations such as matmul, convolution, softmax, normalization, and reductions, PyTorch may dispatch to kernels that have been tuned by expert engineers.

Therefore, beating PyTorch on a generic problem is difficult. The leaderboard becomes interesting because it allows specialization:

- The shapes are known.
- The dtype is known.
- The layout is often known.

- The target GPU is fixed.
- The correctness tolerance is known.
- The input distribution may be known.

A custom kernel can exploit these assumptions while a general library must support a much broader space.

Shape-Specific Optimization

For a fixed matmul shape, a kernel can choose tile sizes exactly for that case. Suppose:

$$M = N = K = 4096.$$

A tiled matmul may use block tiles:

$$M_{\text{tile}} = 128, \quad N_{\text{tile}} = 128, \quad K_{\text{tile}} = 32.$$

The number of output tiles is:

$$\frac{M}{M_{\text{tile}}} \cdot \frac{N}{N_{\text{tile}}} = \frac{4096}{128} \cdot \frac{4096}{128} = 32 \cdot 32 = 1024.$$

Each tile performs:

$$2 \cdot M_{\text{tile}} \cdot N_{\text{tile}} \cdot K$$

FLOPs, where the factor 2 counts multiply and add.

For the chosen tile:

$$2 \cdot 128 \cdot 128 \cdot 4096 = 134{,}217{,}728$$

FLOPs per output tile.

A good implementation tries to maximize data reuse and tensor core utilization for this fixed shape.

Device-Specific Optimization

An H100-specific implementation may exploit features not available or not profitable on older devices. Conversely, a T4-specific implementation may need to be simpler because the device has fewer resources.

For example:

- On H100, tensor core throughput is high enough that feeding tensor cores becomes the challenge.
- On T4, memory bandwidth and smaller caches may dominate.
- On A100, mature tensor core paths and shared memory tiling are important.
- On MI300, AMD-specific programming models and wavefront behavior may matter.

Benchmark Noise

The lecture spends significant time discussing measurement noise. Cloud GPU performance can vary because of:

- thermal conditions,
- power limits,
- machine heterogeneity,
- hypervisor restrictions,
- background load,
- cold start behavior,
- compilation overhead,
- first-run cache effects.

A benchmark should separate compilation from measurement. For Triton, the first invocation may compile the kernel. If that compilation appears inside the timed region, the measured runtime may be much larger than the true steady-state runtime.

A robust benchmark therefore includes warmup:

$$\text{warmup runs} \rightarrow \text{synchronize} \rightarrow \text{timed runs}.$$

Bare Metal as a Source of Truth

The speakers discuss using cloud GPUs through Modal for fast interactive feedback, while also considering bare-metal machines as a more stable source of truth.

The trade-off is:

- Modal gives very fast job startup and convenient short benchmark runs.
- Bare metal gives more control over profiling, counters, power behavior, and noise.

For leaderboard integrity, the ideal system may use both:

$$\text{Interactive cloud feedback} + \text{periodic bare-metal verification}.$$

Profiling and Hardware Counters

The lecture mentions future support for profiling outputs such as Nsight Compute reports. Profiling is essential because runtime alone does not explain why a kernel is slow.

Important profiling quantities include:

- achieved occupancy,
- DRAM throughput,
- L2 hit rate,
- shared memory throughput,
- register usage,
- tensor core utilization,
- warp stall reasons,
- instruction throughput,
- memory transaction count.

A future command such as `/leaderboard submit profile` could provide a profile artifact. This would help participants diagnose bottlenecks rather than guessing from runtime alone.

Correctness and Evaluation Design

Why Correctness is Hard

Correctness is not trivial for GPU kernels because floating-point arithmetic is non-associative:

$$(a + b) + c \neq a + (b + c)$$

in general.

For reductions:

$$y = \sum_{i=1}^{n} x_i,$$

different parallel reduction trees produce different rounding behavior. Therefore, a correct GPU reduction may not exactly match a serial reference.

The evaluator must choose tolerances that accept legitimate parallel implementations but reject wrong outputs.

Reward Hacking

Reward hacking occurs when a submission exploits the test distribution or harness rather than implementing the intended operation.

Examples include:

- Returning zero for a zero-mean vector sum.
- Assuming a fixed seed and hardcoding output patterns.
- Exploiting overly loose tolerances.

- Exploiting shape invariants not intended by the problem designer.
- Triggering undefined behavior that happens to pass on benchmark inputs.

The speakers explicitly encourage participants to find such weaknesses during the practice round because it improves the benchmark.

Public and Private Evaluation

The lecture compares the system to Kaggle-style public and private test sets. Public feedback helps participants iterate, but private cases reduce overfitting to known tests.

A robust evaluation setup can be modeled as:

$$\mathcal{D}_{\text{public}} \cap \mathcal{D}_{\text{private}} = \varnothing.$$

A submission should perform well on both:

$$\text{Correct}\left(f_{\text{sub}}, \mathcal{D}_{\text{public}}\right) = \text{true},$$

and:

$$\text{Correct}\left(f_{\text{sub}}, \mathcal{D}_{\text{private}}\right) = \text{true}.$$

Data Distribution Matters

If input data is drawn from a uniform distribution:

$$A_{ij} \sim \mathcal{U}(0,1), \quad B_{ij} \sim \mathcal{U}(0,1),$$

a participant might exploit the distribution. For example, if only the mean behavior matters under a loose tolerance, the implementation may approximate the output statistically rather than compute it exactly.

For matmul:

$$C_{ij} = \sum_{k=1}^{K} A_{ik} B_{kj}.$$

If A and B are independent uniform random variables on $[0,1]$, then:

$$\mathbb{E}[A_{ik} B_{kj}] = \mathbb{E}[A_{ik}]\mathbb{E}[B_{kj}] = \frac{1}{2} \cdot \frac{1}{2} = \frac{1}{4}.$$

Thus:

$$\mathbb{E}[C_{ij}] = \frac{K}{4}.$$

A bad benchmark with overly loose tolerances might accidentally accept outputs close to this mean. This illustrates why data generation and tolerances are part of benchmark design, not minor details.

Hardware-Level Implications of the Practice Problems

Vector Sum

A vector sum is a reduction:

$$y = \sum_{i=0}^{n-1} x_i.$$

Naively assigning one thread to one element is insufficient because all partial results must be combined. A common GPU strategy is:

1. Each thread loads one or more elements.
2. Threads reduce within a block using shared memory or warp shuffles.
3. Each block writes one partial sum.
4. A second kernel or cooperative strategy reduces partial sums.

The operation has low arithmetic intensity:

$$\text{FLOPs} \approx n, \quad \text{Bytes} \approx 4n, \quad \text{AI} \approx \frac{1}{4}.$$

It is usually memory-bound plus synchronization-bound.

Prefix Sum or Scan

A prefix sum computes:

$$y_i = \sum_{j=0}^{i} x_j.$$

Unlike a simple reduction, it produces n outputs and has a dependency chain. Parallel scan algorithms break this dependency using tree structures.

A common two-phase scan includes:

- upsweep phase to build partial sums,
- downsweep phase to distribute prefixes.

For large inputs, scans require careful handling of block-level partial sums and inter-block prefix propagation.

Matrix Multiplication

Matrix multiplication is central because it can be compute-bound and tensor-core accelerated. The FLOP count is:

$$\text{FLOPs} = 2MNK.$$

The minimum data movement, ignoring cache effects and assuming each matrix is read or written once, is:

$$\text{Bytes} = 4(MK + KN + MN)$$

for FP32.

The ideal arithmetic intensity is:

$$\text{AI} = \frac{2MNK}{4(MK + KN + MN)}.$$

For square matrices where $M = N = K = s$:

$$\text{AI} = \frac{2s^3}{12s^2} = \frac{s}{6}.$$

Thus large matmul can become compute-bound if implemented with sufficient tiling and reuse.

Convolution

The lecture demonstration uses a convolution leaderboard. A direct 2D convolution can be written as:

$$Y_{n,c_{\text{out}},h,w} = \sum_{c_{\text{in}}} \sum_{r} \sum_{s} X_{n,c_{\text{in}},h+r,w+s} W_{c_{\text{out}},c_{\text{in}},r,s}.$$

Convolution optimization depends on:

- input layout, such as NCHW or NHWC,
- filter dimensions,
- stride,
- padding,
- dilation,
- channel count,
- batch size,
- whether the convolution can be lowered to GEMM,
- whether tensor cores can be used.

For small fixed shapes, a custom direct convolution may beat a generic library call if the generic call has overhead or uses a suboptimal algorithm for that specific case.

Leaderboard System Architecture

End-to-End Flow

The complete system can be understood as a pipeline:

Discord command → bot → job scheduler → GPU runner → evaluation harness → result report → leaderboar

The lecture emphasizes that the system is designed to be interactive. Submitting a benchmark should feel much faster than manually renting a machine, installing dependencies, copying files, running scripts, and collecting results.

Why Modal is Used

Modal is used to spin up GPU jobs quickly. The speakers highlight that short benchmark jobs can be cheap and interactive.

The cost model is favorable because most leaderboard evaluations are brief:

$$\text{Total cost} \approx \text{GPU price per second} \cdot \text{seconds per job} \cdot \text{number of jobs}.$$

If each benchmark is only a few seconds, many iterations can be run at low cost.

Discord Bot Channels

The Discord integration includes several channel types:

- general leaderboard discussion,
- task-specific threads,
- submission channels,
- private job threads,
- public leaderboard result threads.

This separation prevents spam and keeps participant feedback organized.

Result Reports

Benchmark output includes information such as:

- benchmark shape,
- correctness status,
- average runtime,
- fastest run,
- slowest run,

- variance or error estimate.

The fastest and slowest values help detect anomalies. For example, if one run is dramatically slower, it may indicate compilation overhead or transient GPU noise.

Common Misconceptions

Misconception: A Correct Kernel is a Fast Kernel

A correct kernel may use poor memory access patterns, unnecessary synchronization, excessive global memory traffic, or too many registers. Correctness is only the first step.

Misconception: PyTorch is Easy to Beat

PyTorch often delegates to heavily optimized vendor libraries. A custom kernel should not be expected to beat PyTorch unless it exploits special constraints such as fixed shape, fixed dtype, fixed layout, or reduced generality.

Misconception: One Kernel Should Be Best on Every GPU

Different GPUs have different bottlenecks. A kernel optimized for H100 may rely on hardware features that do not exist or are less effective on T4.

Misconception: Benchmarking is Just Timing Code

GPU timing requires synchronization, warmup, repeated measurements, and awareness of compilation overhead. Without this, benchmark numbers can be misleading.

Misconception: Loose Tolerances Are Always More User-Friendly

Loose tolerances may admit incorrect or reward-hacked solutions. Tight tolerances may reject legitimate parallel floating-point implementations. Tolerance design is part of benchmark design.

Misconception: Reward Hacking is Only Bad

In a practice round, reward hacking is useful because it exposes weaknesses in the evaluation harness. The goal is to make future benchmark rounds more robust.

Practical Takeaways

For a Kernel Developer

When approaching a leaderboard task:

1. Start with a correct PyTorch implementation.
2. Read the task specification carefully.
3. Identify fixed shapes, dtypes, layouts, and tolerances.
4. Build a simple custom CUDA or Triton kernel.
5. Verify correctness before optimizing.
6. Benchmark repeatedly.
7. Inspect variance, fastest run, and slowest run.
8. Specialize for the target GPU.
9. Use profiling when available.
10. Compare against the reference and current leaderboard times.

For an Evaluation Designer

When designing a benchmark task:

1. Choose commercially or educationally meaningful operators.
2. Define precise input and output types.
3. Use data distributions that are difficult to exploit trivially.
4. Include both public and private tests.
5. Set tolerances based on numerical analysis.
6. Benchmark with warmup and synchronization.
7. Track noise using repeated measurements.
8. Consider bare-metal verification for final rankings.
9. Release strong solutions after the competition to benefit the community.

For AI Kernel Generation

A model that writes kernels must solve a multi-objective problem:

$$\max\left(\text{correctness}, \text{speed}, \text{compilability}, \text{robustness}\right).$$

But these objectives can conflict. For example, a risky low-level optimization may increase speed but reduce portability or correctness.

A good evaluation suite should measure:

- pass rate,
- compile rate,
- runtime,
- speedup over baseline,
- failure modes,
- sensitivity to shape changes,
- sensitivity to input distributions.

Project or Experiment Ideas

Experiment 1: PyTorch Versus Triton Vector Add

Implement vector addition in PyTorch and Triton. Measure runtime for different n. Estimate effective bandwidth:

$$\text{Bandwidth} = \frac{12n}{T}$$

for FP32 vector addition, where T is runtime in seconds.

Compare measured bandwidth to theoretical GPU memory bandwidth.

Experiment 2: Reduction Reward Hacking

Create a vector sum benchmark where:

$$x_i \sim \mathcal{N}(0, 1).$$

Test whether returning zero passes under different tolerances. Then change the distribution to:

$$x_i = \alpha z_i + \beta$$

and observe when the hack fails.

Experiment 3: Naive Matmul Versus Tiled Matmul

Implement:

- naive one-thread-per-output matmul,
- shared-memory tiled matmul,
- tensor-core matmul through Triton or WMMA.

Measure speedup:

$$S = \frac{T_{\text{naive}}}{T_{\text{optimized}}}.$$

Profile global memory throughput, shared memory usage, and achieved occupancy.

Experiment 4: Shape Specialization

Choose a fixed shape such as:

$$M = 4096, \quad N = 4096, \quad K = 4096.$$

Write a specialized kernel for only that shape. Remove unnecessary boundary checks and compare against a generic implementation.

Experiment 5: Benchmark Noise Study

Run the same kernel many times and record:

$$t_1, t_2, \ldots, t_R.$$

Compute:

$$\bar{t}, \quad s, \quad \text{SEM}, \quad \min(t_r), \quad \max(t_r).$$

Observe how noise changes across time, GPU type, and cloud environment.

Experiment 6: Tolerance Sensitivity

For a reduction or matmul, compare FP32, TF32, BF16, and FP16 outputs against an FP32 reference.

Check when:

$$|y_{\text{sub}} - y_{\text{ref}}| \leq \text{atol} + \text{rtol}|y_{\text{ref}}|$$

passes or fails.

This reveals how tolerance choices interact with dtype and accumulation order.

Visual Concept

Draw a Discord message on the left flowing into a leaderboard bot. From the bot, draw arrows to a Modal GPU job runner, then to several hardware boxes labeled T4, L4, A100, H100, and MI300. From each GPU box, draw arrows into an evaluation harness containing input generation, reference output, submitted kernel, correctness check, benchmark timer, and result report. Finally, draw the result flowing back into a public leaderboard table.

Final Mental Model

The lecture's core mental model is:

> A GPU kernel leaderboard is competitive programming for accelerator performance. The problem statement defines the operator and shapes. The evaluation harness defines correctness. The hardware defines the optimization landscape. The benchmark defines the score. The community discovers the tricks, and the released solutions become a public corpus for humans and AI systems to learn high-performance GPU programming.

The practical lesson is that kernel engineering is an end-to-end discipline. It includes CUDA or Triton syntax, but it also includes hardware reasoning, numerical tolerance design, benchmark methodology, cloud infrastructure, profiling, reproducibility, and adversarial thinking about evaluation.

A strong kernel developer asks:

- What exact shape am I optimizing?
- What GPU am I targeting?
- Is the operation memory-bound or compute-bound?
- Are my accesses coalesced?
- Am I using shared memory or tensor cores effectively?
- What is the correctness tolerance?
- Can the benchmark be accidentally gamed?
- Are my timing numbers stable?
- What does the profiler say?

A strong benchmark designer asks:

- Does the reference implementation represent the intended semantics?
- Are the inputs resistant to trivial hacks?
- Are tolerances strict enough to reject wrong outputs but fair to parallel floating-point implementations?
- Are benchmark results stable across machines and repeated runs?
- Can final rankings be verified on less noisy hardware?

The leaderboard matters because it creates a public feedback loop. More people write kernels, better kernels become public, benchmark design improves, and future AI systems get better data for learning how to generate correct and fast GPU code.